

A programozási nyelvek csoportosítása (paradigmák), az egyes csoportokba tartozó nyelvek legfontosabb tulajdonságai

FOGALMAK

- **Programozási nyelv:** egy olyan formális jelölésrendszer, amely számítógépen végrehajtható algoritmusok és adatszerkezetek megvalósítására szolgál
- **Szintaxis:** formai szabályok
- **Szemantika:** működési szabályok, azaz a programok jelentése
- **Paradigma**
 - egy szemléleti mód, mely meghatározza, hogyan látjuk a problémát és hogyan struktúráljuk a megoldást
 - 2 fő csoportja van a paradigmáknak: **imperatív** és **deklaratív**
 - az **imperatív paradigmák** azt írják le, **hogyan kell megoldani egy feladatot** (utasítások sorozatával)
 - állapotváltozások (állapotátmenetek) sorozata
 - a **deklaratív programozás** arra fókuszál, hogy **mi a megoldandó feladat** (specifikáció), azaz milyen tulajdonságokkal rendelkezik a feladat
 - a számítási folyamat itt nem állapotátmenetek sorozata, hanem **kifejezések kiértékelése** vagy **logikai következtetés**
 - 5 fő paradigma van: imperatív(**procedurális, objektumorientált**), deklaratív(**funkcionális, logikai és a párhuzamos programozás**)
- a nyelvek még az **implementációs stratégiájuk** szerint is csoportosíthatók:
 - **fordított (compiled)** - C++
 - a forráskódot egy lépésben gépi kódra fordítjuk
 - **értelmezett (interpreted)** - Python
 - a kódot soronként hajtja végre egy értelmező
 - **hibrid** - JAVA
 - a kód köztes kódra, azaz byte kódra fordul, amit egy virtuális gép futtat

IMPERATÍV, PROCEDURÁLIS PARADIGMA

- központjában az **állapotváltozások** és az **utasítások szekvenciális végrehajtása** áll
 - procedurális programozás annyival bővíti ki ezt, hogy az utasításokat procedurákba (eljárások, függvények) csoportosítja
- a programozó azt írja le, hogy a számítógépnek pontosan hogyan kell megoldania a feladatot, lépésről lépésre utasítva azt az állapot (a memória tartalmának) módosítására
- **Jellemzői**
 - **változók és értékadás**
 - a változók memóriacímek absztrakciói
 - pointer típus jellemző az ilyen nyelvekre
 - legfontosabb művelet az értékadás ($x := x + 1$), mely felülírja a régi állapotot

- **vezérlési szerkezetek**
 - végrehajtás szekvenciális, feltételes elágazásokkal és ciklusokkal irányítunk
- **hatékonyság**
 - mivel ez áll legközelebb a hardver működéséhez általában ez a leghatékonyabb
- **Imperatív személelet fejlődési szakaszai**
 1. **gépi kód és assembly**
 - közvetlen hardverközelí utasítások, ahol az adatok még szabadon védelem nélkül érhetők el
 - nehézkes és nem hordozható
 2. **procedurális programozás (proceduralitás) - C, Pascal**
 - megjelennek az alprogramok (eljárások, függvények) melyek segítik a kód strukturálását és újrafelhasználását
 - az összetett feladatokat egyszerűbb utasításokra (pl. értékadás, alprogramhívás) és vezérlési szerkezetekre (szekvencia, feltételes utasítás, ismétlés) bontja
 3. **moduláris programozás**
 - a programot önálló, jól definiált egységekre (modulokra) bontják
 - feladat részfeladatokra bontása (dekompozíció)
 - meglévő egységeket használunk fel újra (kompozíció)
 - alapelve az explicit interfészek használata és a modulok közötti gyenge kapcsolat fenntartása
 4. **absztrakt adattípusok**
 - az adatok és a műveletek szorosabb egysége (**egységbezárás, encapsulation**), az **információ elrejtésének** kezdete
 - ez a koncepció a moduláris programozás kulcsa
 - alapegysége a **modul**, a modulok önálló egységek és a saját feladatukat látják el
 - **interfész (csatlakozási felület)**, ez az exportált komponensek összessége, amit a felhasználó lát
 - meghatározza mit csinál a modul
 - **megvalósítás (implementáció)**, a felhasználó elől elrejtett rész megvalósítása, mely bármikor módosítható anélkül, hogy modult használó kód elromlana
 - meghatározza hogyan működik a modul
 5. **objektumorientáltság**
 - a viselkedésminták, példányosítás és öröklődés megjelenése

C nyelv

- mintaképe az imperatív paradigmának, mivel a program végrehajtását állapotátmenetek sorozataként kezeli

- **utasítás és állapotközpontúság**
 - a program alapját értékadások és vezérlési szerkezetek (ciklusok, elágazások) adják
 - ezek végrehajtása közvetlenül módosítja a memóriában tárolt állapotot
- **procedurális absztrakció**
 - az utasításokat függvényekbe csoportosítjuk, így egy összetett algoritmust logikai részekre tudjuk bontani
 - ez újrafelhasználhatóságot és logikai csoportosítást ad
 - a függvények bemenő adatokból állítanak elő egy kimenő adatot
- **pointerek és memóriakezelés**
 - a mutatók (pointerek) közvetlen hozzáférést biztosítanak a memóriacímekhez, így az adatok hatékonyan módosíthatók
 - programzó maga felügyeli a memóriefoglalást és a változók élettartalmát
- **mellékhatás (side effects)**
 - egy művelet kiszámítása során külső változók értéke is módosulhat
 - ez bonyolítja a kód követését, elengedhetetlen a hatékony erőforrás-kezeléshez

OBJEKTUMORIENTÁLT PARADIGMA (OOP)

- olyan szemléleti mód, mely a szoftverfejlesztést modellezésként fogja fel, tehát nem utasítások listájaként, hanem egymással kommunikáló önálló egységekként (objektumok) kezeli
- **Osztályok**
 - egy **absztrakt adattípus**, a felhasználó által definiált adat, ami egy tervrajzként / sablonként működik
 - egy osztály leírja azonos attribútumokkal és metódusokkal rendelkező objektumok egy csoportját
 - két fő komponense van
 - **attribútumok (mezők):** objektum belső állapotai
 - **metódusok (tagfüggvények):** az objektumok viselkedését definiálják
- **Objektumok (példány)**
 - az objektumok az osztályok példányai
 - objektum az osztály alapján a memóriában létrejött konkrét entitás
 - minden objektum rendelkezik 3 dologgal:
 - **állapot:** attribútumok, mezők aktuális értékei
 - **viselkedés:** metódusok, azaz, hogy reagál bizonyos kérésekre
 - **identitás:** minden objektum egyedi, azaz megkülönböztethető más objektumoktól (memóriacím vagy referencia alapján)
 - még ha állapotai egyeznek is két objektumnak azok akkor is különbözőnek számítanak
- az objektumok **metódusok** hívásával kommunikálnak egymással
- 4 alappillére épül: **egységbezárás, absztrakció, öröklődés és polimorfizmus**
- ilyen nyelvek: JAVA, C++, Smalltalk

Egységbezárás (encapsulation)

- egységbezárás azt jelenti, hogy az adatokat (attribútumokat) és a rajtuk végzett műveleteket (metódusokat) egy logikai egységbe (osztály) kezeljük
- fő célja az **információ elrejtés**
 - az objektum belső állapota nem érhető el közvetlenül a külvilág számára csak vezérelt módon, publikus metódusokon keresztül
 - publikus getter/setter és private attribútumok
 - az információ elrejtés biztosítja az integritást
 - pl: nem lehet érvénytelen adatot beállítani (pl: negatív életkor), mert a setter ellenőrizheti a bemenetet
 - ezt a **láthatósági (hozzáférési) módosítókkal** érjük el
 - **private**: csak az osztályon belül látható
 - ajánlott szintje az adatoknak
 - **default** (package-private) - CSAK JAVA-ban
 - csak az azonos csomagban (package) lévő osztályok látják
 - **protected**: a csomagon belül és az osztályon belül, illetve a gyermekosztályok számára látható (akár más csomagban lévő gyermekosztályok számára is)
 - **public**: bárki számára elérhető
 - ajánlott szintje a metódusoknak
 - előnye, hogy ha a belső megvalósítás meg is változik akkor is a külvilág felé mutatott interfész változatlan maradhat és a többi programrészben nem kell módosításokat végezni

Absztrakció

- célja, hogy a felesleges dolgok, mint a megvalósítás eltűnjön
 - azaz segít fókuszálni arra, hogy mit csinál egy entitás, ahelyett, hogy arra fókuszálna, hogy hogyan csinálja azt
- az **interfészek** és **absztrakt osztályok** valósítják ezt meg
 - az **interfész** segítségével egy osztály váza írható le, azaz, hogy milyen metódusokkal rendelkezik, anélkül, hogy ezek implementásra kerülnének
 - metódusokat definiálnak, amiket más osztályok implementálhatnak
 - ha az adott osztályokba legalább 1 metódusa az interfésznek nem kerül implementálásra, akkor az osztály absztrakt lesz
 - egy osztály több interfészt is implementálhat
 - az absztrakt osztályok **olyan osztályok, amik nem példányosíthatók**, és tartalmazhatnak absztrakt metódusokat (kód nélküli deklarációkat)
 - amennyiben egy osztálynak van 1 absztrakt metódusa akkor az osztály automatikusan abstract lesz
 - céljuk, hogy ösként szolgáljanak a származtatáshoz, kikényszerítve bizonyos metódusok megvalósítását a leszármazottakban

Öröklődés

- az öröklődés lehetővé teszi, hogy meglévő osztályokból (ősosztály, szülő) új osztályokat (származtatott osztály, gyerek) hozzunk létre
 - a gyerekosztály örökli a szülő tulajdonságait (attribútumok) és viselkedését (metódusok), ezeket kiegészítheti vagy módosíthatja is
 - az örökölt dolgok megvalósítását felülírhatjuk (polimorfizmus), illetve a láthatóságait is módosíthatjuk de csak szigorítási formába (ha public volt akkor lehet protected és private is, de ha private volt akkor se public, se protected)
- célja a **kód újrafelhasználása** és a **hierarchikus rendezés**

Polimorfizmus

- azt jelenti, hogy különböző típusú objektumokat (pl. Kutya, Macska) közös ősök (pl. Állat) típusán keresztül tudunk kezelni, és az objektumok a saját típusuknak megfelelően fognak viselkedni
- segít, hogy a kód flexibilis és újrahasználható legyen
- **Statikus polimorfizmus (korai hozzárendelés)**
 - **metódus túlterhelés (overload)**
 - ugyanazon osztályon belül több metódusnak is ugyanaz a neve, de a paraméterlistájuk (típus vagy számosság) különböző
 - pl: számol(int a, int b) és számol(double a, double b).
 - fordító már **fordítási időben** tudja melyik metódus kerül meghívásra, a paraméterek alapján
- **Dinamikus polimorfizmus (késői hozzárendelés)**
 - **metódus felülírás (override)**
 - gyerekosztály felülírhatja az ősosztályának metódusait
 - itt a paraméterlista nem változik
 - lehetővé teszi, hogy a különböző osztályok eltérő viselkedését ugyanazon az interfészen keresztül használjuk
 - példa
 - Állat ősosztály hangotAd() metódusát, a Kutya felülírja (vau), a Macska felülírja (miau)
 - *Kód logikája*
 - létrehozol egy listát, amiben Állat referenciák vannak
 - belepakolsz egy kutyát és egy macskát, majd végigmész a listán és mindegyiknek kiadod a parancsot: hangotAd()
 - *Eredmény:* Bár a listában "csak" állatokként látod őket, a kutya ugatni fog, a macska nyávogni, ez a polimorfizmus: **nem kell tudnod, pontosan mi az, csak azt, hogy egy Állat, és tud hangot adni.**
 - **futási időben** történik, híváskor dől el pontosan melyik metódus kerül meghívásra
 - nem a fordító dönt arról melyik metódus fusson le

Smalltalk

- gyengén típusos
- objektum orientált nyelv
 - minden osztály közös őse az Object osztály
 - egyszeres öröklődés, osztály (static) változók, metódusok (üzenetek)
 - minden metódus virtuális és publikus
 - minden adattag privát objektum szintű védelemmel
 - konstruktor = new üzenet (ezt a classhoz kell extend-elni)
 - magára a self-el, szülőjére a super-el hivatkozhatunk
- értékadás visszaadja a kapott értéket
- Utasítások és üzenetek
 - vannak programrészek, emiatt lokális és globális változók is
 - az utasításokat . vagy ! segítségével zárjuk
 - különbség, hogy !-el programrészt zárunk le (nem globális változók eldobása)
 - üzenetek típusai
 - unáris (paraméter nélküli)
 - balról jobbra asszociatív
 - *<objektum> <utasítás, üzenet>*
 - bináris (aritmetikai műveletek)
 - *<objektum> <operátor> <objektum>*
 - kulcsszavas (több paraméteres függvénynek feleltethető meg)
 - mindig zárójelezzük az asszociativitásnál
 - *<objektum> <kulcsszó_1>: <arg_1> <kulcsszó_2>: <arg_2>*
 - üzenetek precedenciája (végrehajtási sorrend)
 - azonos precedencia esetén, mikor zárójel nincs, akkor balról jobbra értékeljük ki
 - zárójel - unáris - bináris - kulcsszava
 - Típusok
 - Integer, Karakter (\$a, \$\$, \$'), String: 'ez string'
 - Blokk: [1. 2. 3.]
 - Tömb: #(1 'xyz' (2 3) \$f)
 - Változók
 - lokális változók, ezek egy adott programrészen belül élnek csak
 - !-es lezárás után már nem érhetők el

```
| változo_1 | " változo deklarálás "  
| val1 val2 | " lehet többet is "  
változo_1 := 14. " változohoz értéket, adok, itt := az értékadás "  
változo_1 _ 15. " _ is értékadás "  
val1 := val2 := 5. " összefuza is megadható val1 = 5 es val2 = 5 "  
változo_1 displayNL.  
"Hello" printNL.  
  
változo_2 := 12 " így is megadható, de általában am szokás "  
  
#( 1 2 3) size  
#( 1 2 3) copy  
#( 1 2 3) at: 1  
#( 1 2 3) copy at: 1 put: 4  
  
" Kaszkád: egy objektumnak több üzenet "  
5 + 3; + 1; + 7  
  
nil " objektum konstans, pl nem létező változó kiírása esetén ezt adja vissza "
```

- globális változók

- programrész lezárás után (!) sem dobódnak el automatikusan a változók

```
Smalltalk at: #x put: 500! "kulcsszavas módon lehet csak létrehozni"  
| y |  
y := 15.  
x max: y displayNl.
```

- programblokk

```
| x y z |  
x := [1. 2. 3.].  
x value printNl. " value megadása, kell különben csak Block Closures ir "  
y := [1. 2. 3.] value.  
y printNl.  
z := [] value. # ures blokk eseten nil  
  
[  
  5.  
  3*2.  
  'hello'.  
] value printNl. " a blokk visszaküldi az utolsó utasítás visszatérési értékét "  
  
[:a :b | " a blokk tartományában értelmezett (lokális) változók "  
  (a+b) printNl. " 8 "  
] value: 3 value: 5 " paraméterezése a blokknak (a és b lokális változónak ad értéket) "
```

- Osztályok létrehozása és új metódusok hozzáadása

```
Number subclass: Tort [  
  tortekSzama := 0. " statikus adattag, magához az osztályhoz tartozik "  
  
  | adattag1 adattag2 | " példányváltozók, adattagok "  
  
  getAdattag1 [ " getter "  
    ^adattag1.  
  ]  
  
  setAdattag1: a1 setAdattag2: a2 [ " setter "  
    adattag1 := a1.  
    adattag2 := a2.  
  ]  
  
  sajátMetódusom [  
    ...  
  ]  
  
  + másik [  
    | eredmény |  
    eredmény := adattag1 + másik getAdattag1.  
    ^eredmény.  
  ]  
  
  printOn: stream [ " toString "  
    ' ezt kiírom: ' displayOn: stream.  
    adattag1 displayOn: stream.  
  ]  
]
```

```
Tort class extend [  
  new [ " parameter nélküli konstruktor "  
    | obj |  
    obj := super new. " os objektum létrehozása "  
    obj setAdattag1: 1 setAdattag2: 1.  
    tortekSzama := tortekSzama + 1. " statikus adattag módosítása "  
    ^obj.  
  ]  
  
  new: a1 new: a2 [ " parameteres konstruktor "  
    | obj |  
    obj := super new.  
    obj setAdattag1: a1 setAdattag2: a2.  
    ^obj.  
  ]  
  
  getStatikusAdattag [  
    ^(statikusAdattag).  
  ]  
]
```

FUNKCIONÁLIS PARADIGMA (APPLIKATÍV)

- matematikai alapja a λ -kalkulus
- a program függvények, kifejezések kiértékelésének folyamata
- **Jellemzői**
 - **állapotmentesség**
 - változók értéke a hozzárendelés után nem változik meg, nincs hagyományos értelemben vett értékadás
 - ha $x = 5$, akkor x értéke örökre 5 marad a hatókörön belül
 - **referencia szerinti átlátszóság**
 - egy függvény ugyanarra a bemenetre mindig ugyanazt a kimenetet adja, független attól, hogy mikor hívjuk meg
 - megkönnyíti a tesztelést és a párhuzamosítást
 - **rekurzió**
 - ciklusok (for, while) helyett rekurziót használunk a feladatok ismétlésére
 - **magasabb rendű függvények**
 - a függvények átadhatók paraméterként más függvényeknek és akár visszatérési értékek is lehetnek (pl: map, filter)
 - **lusta kiértékelés**
 - normalizáló kiértékelés, mindig megtalálja a normálformát, ha létezik
 - egy kifejezés csak akkor értékelődik ki amikor az eredményére tényleg szükség van
 - lehetővé teszi a végtelen adatszerkezetek (pl: végtelen lista) kezelését

Haskell

- **program mint függvény**
 - a Haskell programokat úgy tervezték, hogy maguk is függvények legyenek
 - egy Haskell program futtatásakor egy meghatározott belépési pontba kerülünk, ahonnan a függvények sorozata hajtódik végre
 - egyszerű függvény
 - **mintaillesztés**
add :: Int -> Int -> Int
add x y = x + y
add 5 y = 5
 - **örök**
max :: Int -> Int -> Int
max x y
| x >= y = x
| otherwise = y
 - **feltételes kifejezés**
max :: Int -> Int -> Int
max x y = if x > y then x else y
- a változók nem tárolnak állapotot, hanem konkrét értékeket reprezentálnak

- **rekurzió**
 - ciklusok helyett rekurzió van az ismétlődő feladatok megoldására
 - `factorial :: Int -> Int`
`factorial 0 = 1` //bázis eset
`factorial n = n * factorial (n - 1)` //rekurzív eset
- **lusta kiértékelés**
- **erős és statikus típusosság**
 - a fordító szigorúan ellenőrzi a típusokat, de gyakran képes magától is kitalálni azokat (típuskövetkeztetés)
- **mintaillesztés**
 - a függvények definíciója során közvetlen leírhatjuk az adat szerkezetét
- **magasabb rendű függvények**
 - a függvények átadhatók paraméterként más függvényeknek és akár visszatérési értékek is lehetnek (pl: `map`, `filter`)
- **listák (head:tail forma)**

```
[1, 2, 3, 4]
[[1, 2], [3], [4, 5, 6]] --listában lehet listákat is tarolni
[1, 3 .. 20] --automatikus feltöltés, itt 1 és 20 között a paratlan számokat tartalmazza
[1..] --vegtelen lista
['a' .. 'z'] --kis- és nagybetűket tartalmazó String lista

[1, 2] ++ [3, 4, 5] --összefűzés
5 : [1, 2] --elem hozzáadás [5, 1, 2]

length [1, 2, 3] --lista hossza, itt 3
null [1, 2, 3] --üres-e a lista, True vagy False
[1, 2, 3] !! 2 --listából elem lekérése ami az adott indexen van (0-tól indexelünk)
reverse [1, 2, (-3), 4, 5] --eredménye [5,4,-3,2,1]

asd :: [Int] -> Int
asd [] = 0
asd (headElem:tailElem) = headElem + asd tailElem
-- h:t tombok esetén annyit jelent, hogy a h lesz az első eleme a t pedig a h elem nélküli tomb

--map egy függvényt alkalmaz a lista minden elemére
map (+1) [1, 2, 3] --eredménye [2,3,4]
map (`mod` 2) [1, 2, 3] --eredménye [1,0,1]

--filter visszaadja az összes olyan elemet a listából egy listaként, ami teljesíti a megadott feltételt
filter (>2) [1, 2, 3] --eredménye [3]
filter (even) [1, 2, 3] --eredménye [2]

--zip: összekapcsol két lista elemeit páronként
zip [1, 2, 8, 9] [5, 6, 7] --eredménye [(1,5),(2,6),(8,7)], aminek a masikbol mar nincs parja kihagyja

--foldr: egyesíti a lista elemeit a megadott függvény szerint
foldr (+) 5 [1, 2] --eredménye 8 (5 + 1 + 2)
```

LOGIKAI PARADIGMA (SZABÁLY ALAPÚ)

- matematikai logikán alapul
- a program tények és szabályok halmaza, a futtatás pedig egy tételbizonyítási folyamat
- **Felépítése**
 - **termek**, értékek
 - pl: janos, 5, alma
 - **tények**
 - igaznak tekintett állítások az objektumokról és kapcsolataikról
 - pl: szuloje(janos, bori)
 - **szabályok**
 - “ha-akkor” típusú következtetési szabályok
 - pl: nagyszuloje(X, Z) :- szuloje (X, Y), szuloje (Y, Z)
 - **lekérdezés**
 - a felhasználó kérdéseket tesz fel, amit a rendszer próbál bizonyítani
 - pl: nagyszuloje(janos, Ki);
- klóz = adatbázis elemei (tények + szabályok)
- **Működése**
 - a rendszer beépített következtető motorja keresi a válaszokat
 - **unifikáció (illesztés)**: két kifejezés illesztése a változók megfelelő behelyettesítésével
 - **rezolúció**: a következtetési lépések láncolata
 - **visszalépéses keresés (backtracking)**: ha egy ágon elakad a keresés, a rendszer visszalép és új alternatívát próbál

Prolog

- ideális mesterséges intelligencia feladatokra és szakértői rendszerekhez
- **adatbázis-szemlélet**
 - a program lényegében egy tudásbázis, amelyben relációkat (pl a szuloje reláció) tárolunk
- **nem-determinisztikus**
 - egy kérdésre több válasz is születhet, amit a rendszer sorban képes előállítani

```
alma; // alma term letrehozasa
mary; // mary term letrehozasa
joe; // joe term letrehozasa

likes(mary, alma); // relacio letrehozasa a ket term kozott

?- likes(mary, alma); // kerdes feltetele a programnak
> True. // az eldontorendszer valasza

?- likes(joe, alma); // ujabbi kerdes
> False. // false, mivel nem talalt ilyen kapcsolatot a
// rendszer

?- likes(X, alma); // ki szereti az almat? kerdes feltetele
> mary. // az adatbazis vizsgalata utan erre a dontesre
// jutott a rendszer
```

PÁRHUZAMOS PARADIGMA

- a párhuzamos programozás célja, több utasításozrat egyidejű végrehajtása, a hardver (többmagos processzorok, elosztott rendszerek) jobb kihasználása érdekében
- **Jellemző problémái a párhuzamos programozásnak**
 - **holtpont (deadlock)**
 - folyamatok kölcsönösen egymásra várnak, ilyenkor a rendszer megáll
 - **kiéheztetés**
 - egy folyamat sosem jut hozzá a szükséges erőforráshoz, bár nincs holtpont
 - mindig a legmagasabb prioritású folyamat fut
 - pl: kórházban a sürgősségin a súlyosabb sérült előbb kerül sorra, mint aki csak lázas
 - megoldás rá az **aging** (öregítés), azaz minnél tovább vár egy folyamat annál inkább nő a prioritása
 - **nem-determinizmus**
 - a folyamatok relatív sebessége miatt ugyanaz a program ugyanarra a bemenetre más kimenetet adhat
 - ez megnehezíti a hibakeresést
- 2 féle modellje van: **közös memória** és **osztott memória**

Közös memória

- a szálak közös címtartományon osztoznak, ilyenek pl: JAVA és C# szálak
 - JAVA-ban is vannak szálak a Thread osztály példányosításával, vagy a Runnable interfész implementálásával és synchronized blokkok, valamint a wait() és notify() metódusok valósítják meg a monitor-szemléletet
- kritikus a szinkronizáció a versenyhelyzet elkerülésére
- **kölcsönös kizárás**
 - **egyszerre csak egy folyamat lehet kritikus szekcióban**
 - kritikus szekció a közös, megosztott erőforrást kezelő kódrészlet egy folyamatban
- szinkronizáció megvalósíthatása: **szemaforok**, **mutex-ek** vagy **monitorok**
- **Szemafor**
 - a szemafor egy nem negatív egész változó melyben számolhatjuk az ébresztéseket a későbbi felhasználás céljából
 - használjuk **kölcsönös kizárásra** és **szinkronizációra** (sorrendiségre) is
 - 2 atomi (megszakíthatatlan, azaz addig még nem fejeződnek be addig a szemafort más folyamat nem éri el) művelete van:
 - **down (vagy P, Wait)**
 - csökkenti az értékét
 - ha érték 0-nál nagyobb volt akkor a folyamat továbbhalad
 - ha 0 volt az érték akkor a folyamat alvó állapotba kerül, azaz blokkolódik anélkül, hogy befejezné a műveletet

- **up (vagy V, Signal)**
 - növeli a szemfor értékét
 - ha volt alvó folyamat ezen a szemaforon akkor egyet felébreszt, ami befejezheti a saját down műveletét
- mikor használjuk?
 - készletkezelés, például egy erőforrásból több is van
 - jelzés (signaling), ha az egyik folyamatnak szólania kell a másiknak mikor kész van, hogy a másik folytassa
- megoldja a termelő-fogyasztó hibáját, mert emlékszik a jelekre
 - ha a termelő küld egy up-ot, az akkor is növeli a szemafor értékét, ha senki se alszik rajta, így a fogyasztó később látni fogja, hogy van dolga
- példa
 - egy könyvtár, ahol egy könyvből 5 példány van, itt a szemafor értéke kezdetben 5
 - mindenki, aki kivesz egyet, végrehajt egy down műveletet (az érték csökken), ha elfogyott (érték = 0), a következő olvasónak várnia kell a könyvtárban (elalszik)
 - amikor valaki visszahozza, up műveletet végez, és az érték nő, a várakozó pedig megkapja a könyvet
- **Mutex (bináris szemafor)**
 - szemafor egy speciális, egyszerűsített változata, amit csak kölcsönös kizárásra használunk
 - akkor használjuk ha van egy darab közös erőforrás és biztosítani akarjuk, hogy egyszerre csak egy folyamat férjen hozzá
 - értéke csak bináris lehet (2 állapota)
 - 0 (nyitott, szabad)
 - 1 (zárt, foglalt)
 - használata:
 - mutex_lock
 - ha a mutex szabad (értéke 0), akkor lezárja (értéke 1 lesz) és a folyamat belép a kritikus szekcióba
 - ha pedig a mutex foglalt (értéke 1) akkor a hívó blokkolódik
 - mutex_unlock
 - feloldja a zárat és ha van várakozó, felébreszti
 - **a mutexnek van gazdája, csak az oldhatja fel aki lezárta**
 - példa
 - van egy mosdó, ennek a kulcsa a mutex
 - aki be akar menni, elkéri a kulcsot (lock), ha a kulcs ott van, beviszi, magára zárja az ajtót
 - aki utána jön, látja, hogy nincs kulcs, ezért leül és vár (elalszik)
 - amikor az első kijön, visszateszi a kulcsot (unlock), és szól a várakozónak, hogy mehet
- TSL vs Mutex
 - TSL tevékenyen várakozik, ezzel égeti a CPU-t, míg a Mutex viszont elaltatja a folyamatot, ezzel a CPU-t átadja másik folyamatnak

- **Monitor**

- a szemaforok használta nehéz, könnyen el lehet rontani a sorrendet, ami holdponthoz vezet, ehelyett a **monitor egy magasszintű programozási nyelv konstrukció** (JAVA-ban a synchronized)
- a monitor egy modul, osztály, mely adatokat és eljárásokat tartalmaz
 - az eljárásai automatikusan kölcsönös kizárással futnak, azaz egyszerre csak egy processzus tartózkodhat aktívan a monitoron belül, ha egy másik is hívná valamely eljárását a monitornak akkor várakoznia kell
- vannak **feltételi változók**, amik a szinkronizációt segítik, ezek abban segítenek, hogy a monitoron belül is lehessen várni (pl: üres buffer esetén) 2 művelet van:
 - wait(c)
 - a folyamat elalszik a c feltételen és ideiglenesen elhagyja a monitort, hogy más bejöhessen
 - signal(c)
 - felébreszti az egyik folyamatot, mely a c feltételre vár

Osztott memória

- a folyamatok saját memóriával rendelkeznek és üzenetküldéssel kommunikálnak
- **üzenetküldés**
 - az üzenetküldés az egyetlen mód a kölcsönös kizárásra elosztott rendszerekben, **ahol nincs közös memória**, mert a folyamatok különálló, hálózaton összekapcsolt gépeken futnak
 - működése
 - a folyamatok struktúrált adatsomagokat, üzeneteket küldenek az operációs rendszeren keresztül
 - hasonlít a szemaforokhoz, de az üzenetek nem csak jeleznek, hanem adatokat is szállítanak
 - itt a programozónak explicit módon hívnia kell a küldő/fogadó műveleteket, a primitíveket vagy rendszerhívásokat:
 - send(destination, message)
 - üzenet küldése egy célállomásnak, mely lehet egy folyamat azonosítója, vagy akár egy köztes tároló is
 - receive(source, message)
 - üzenet fogadása, egy adott forrástól, a hívó általában blokkolódik, ameddig meg nem érkezik az üzenet
 - problémái:
 - elveszett üzenetek
 - a hálózaton az üzenet elveszthet
 - megoldás rá a nyugtázás, azaz ha a fogadó egy **nyugtázó üzenetet (ACK)** küld vissza és ha a küldő egy bizonyos időkorláton (timeout) belül nem kapja ezt meg akkor újraküldi

- azonosítás / címzés
 - lehet **közvetlen**, mikor folyamat ID-re címezzük
 - lehet **közvetett**, mikor például egy postaládán keresztül küldjük
- hitelesítés
 - biztonsági szempont, azaz biztosnak kell lenni benne, hogy a megfelelő folyamat kapja meg az üzenetet, illetve, hogy a feladó is az legyen akinek mondja magát
- teljesítmény
 - üzenetkezelés általában lassabb mint a többi IPC (szemafor, mutex) mert adatok másolásával és hálózati késletetéssel jár
- **Közvetett kommunikáció (levelesláda)**
 - ahelyett, hogy közvetlenül folyamatoknak küldenénk az adatot, inkább egy levelesláda adatszerkezetet használunk, mely adott számú üzenet tárolására alkalmas
 - küldő a levelesládába teszi az üzenetet, a fogadó pedig onnan veszi ki
 - mikor a levelesláda tele van akkor a küldő blokkolódik, ha pedig üres akkor a fogadó fog blokkolódni
- **Közvetlen kommunikáció (randevú)**
 - ha elhagyjuk az ideiglenes tárolást (postaláda) akkor a küldő és fogadó közvetlenül találkozik
 - mint egy fizikai találkozó, ha a küldő folyamat ér oda előbb az üzenettel, meg kell várja a fogadó folyamatot, ha pedig a fogadó ér oda előbb akkor meg kell várni a küldő folyamatot mely hozza az üzenetet
 - amikor mindkettő folyamat odaér akkor az adatátadás megtörténik és mindkét folyamat megy tovább
 - előnye a könnyű megvalósítás, mivel nincs szükség memória kezelésre
 - hátránya, hogy kevésbé rugalmas, szorosabban kapcsolódnak egymáshoz a folyamatok

Occam nyelv

- a folyamatok saját memóriával rendelkeznek és csatornákon (channel) keresztül kommunikálnak
 - alapjai: változók, folyamatok, csatornák
- **csatornák**
 - egyirányú, szinkron kapcsolatok, ahol a küldő és a fogadó bevárja egymást az adatátvitelhez
 - 1 küldő és 1 fogadó
 - biztonságos
 - csatorna deklaráció: CHAN OF INT csatornam
 - input (csatornából kiolvasás): csatornam ? valtzm
 - output (csatornába írás): csatornam ! 5
- folyamatok élelciklusa
 - elindul
 - fut
 - befejeződik
 - nem mindig, mivel holtpontra is kerülhet
- elemi folyamatok
 - üres utasítás: SKIP
 - élelciklusa: elindul, nem csinál semmit, befejeződik
 - bépített holtpontra: STOP
 - élelciklusa: elindul, nem csinál semmit, nem fejeződik be
 - egyszerű értékadás: változom := 0
 - többször értékadás x, y := 1, 2
- alapvető vezérlési szerkezetek
 - **SEQ:** szekvenciális végrehajtás
 - **PAR:** utasítások párhuzamos végrehajtás
 - küldő nem vár a fogadóra (asszinkron végrehajtás)
 - lehetnek már elküldött, de még nem fogadott, feldolgozott üzenetek
 - pufferekt csatorna
 - csatorna kapacitás is van
 - korlátlan vagy korlátos
 - **ALT:** alternatív választás (amelyik csatornán előbb jön adat)
 - occam várakozás
 - addig vár, amíg valamelyik feltétel igaz nem lesz, ekkor az ehhez tartozó folyamat lefut, majd az ALT befejeződik
- Multiplexer
 - több forrásból érkező adatok átirányítása egy kimenő csatornára
 - ALT nélkülözhetetlen a megvalósításához
- Demultiplexer
 - bejövő csatornán érkező adatok szétválogatása kimenő csatornákra
 - SEQ nélkülözhetetlen a megvalósításához